SS

BACKEND GUIDE · NODE + EXPRESS + MONGODB

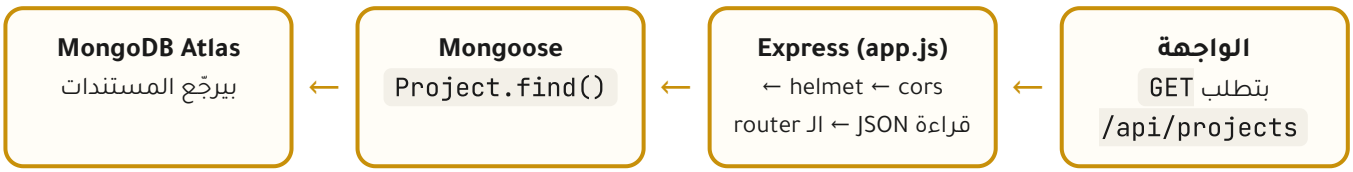
دليل الباك إند
بورتفوليو سالم صالح

شرح مبسّط للسيرفر اللي وراه الموقع: إزاي الطلب بيوصل ويترد عليه، وإزاي البيانات بتتخزن في MongoDB، وإزاي لوحة التحكم محمية، وكل حماية موجودة وسبب وجودها – وإزاي تضيف مورد جديد خطوة بخطوة.

1. الصورة الكبيرة
2. التشغيل والإعدادات (`.env`)
3. خريطة الملفات
4. أساسيات HTTP و REST
5. رحلة الطلب في app.js
6. قاعدة البيانات (Mongoose)
7. تسجيل الدخول (JWT)
8. الحماية وأسبابها
9. كل عناوين الـ API
10. الإيميل وإبقاء السيرفر صاحي
11. النشر على Render و Atlas
12. إضافة مورد جديد
13. مشاكل حصلت واتعلمنا منها
14. قاموس المصطلحات

1 الصورة الكبيرة

السيرفر برنامج **Node.js** مكتوب بـ **Express**، شغّال على **Render**. شغلته يستقبل طلبات من الواجهة، ويقرا أو يكتب في قاعدة البيانات **MongoDB Atlas**، ويرد بـ **JSON**. هو مش بيعرض صفحات – الصفحات كلها في الواجهة (React).



والرد بيرجع بنفس الطريق بالعكس: JSON فيه قايمة المشاريع.

الأدوات

الحزمة	دورها
`express`	إطار السيرفر: الراوتس والـ middleware.
`mongoose`	التعامل مع MongoDB بـ schemas و models.
`jsonwebtoken` + `bcryptjs`	تسجيل دخول الأدمن (توكن + تشفير الباسورد).
`helmet` , `cors` , `express-rate-limit`	الحماية (هيتشرحوا في الفصل 8).
`express-async-errors`	أي خطأ في دالة async يروح لمعالج الأخطاء لوحده.
`nodemailer`	إرسال إيميلات الإشعار (SMTP).
`dotenv`	قراءة الإعدادات من ملف `.env` .

2 التشغيل والإعدادات (`.env`)

```
cd server
npm install
npm run dev       # بـ nodemon: بيعيد التشغيل لوحده مع أي تعديل
npm start         # تشغيل عادي (ده اللي Render بيستخدمه)
npm run seed      # يملا قاعدة بيانات فاضية ببيانات تجريبية + حساب الأدمن
```

> **تحذير مهم عن seed**
>
> `npm run seed` **بيمسح** كل المشاريع والمهارات والفيديوهات قبل ما يضيف التجريبية. استخدمه على قاعدة بيانات جديدة بس – مش على اللي فيها محتواك.

متغيرات البيئة

الإعدادات السرية أو اللي بتختلف بين جهازك والسيرفر الحقيقي ما بتتكتبش في الكود. على جهازك في `server/.env` (متجاهَل في git)، وعلى Render في صفحة Environment.

المتغير	معناه
`MONGODB_URI`	عنوان قاعدة البيانات (محلي على جهازك، Atlas على Render). **إجباري.**
`JWT_SECRET`	السر اللي بيوقّع توكنات الدخول. **إجباري** ولازم يكون طويل وعشوائي.
`PORT`	البورت (4000 محليًا؛ Render بيحدده لوحده).
`CLIENT_ORIGIN`	الموقع الوحيد المسموح له يكلم السيرفر (CORS): `https://salemsalehtech.github.io`
`ADMIN_EMAIL` / `ADMIN_PASSWORD`	بيستخدمهم `seed` عشان يعمل حساب الأدمن.
`NOTIFY_EMAIL` + `SMTP_*`	إيميلات الإشعار (الفصل 10).
`RENDER_EXTERNAL_URL`	Render بيحطه لوحده – بيستخدمه الـ keep-alive.

لو `MONGODB_URI` أو `JWT_SECRET` ناقصين، `server.js` بيوقف فورًا برسالة واضحة – أحسن من سيرفر "شغّال نص شغل".

3 خريطة الملفات

```
server/
├── package.json                ← الحزم والأوامر (dev, start, seed)
├── .env.example                ← نموذج للإعدادات (من غير أسرار)
├── prisma/seed.js              ← بيانات تجريبية + حساب الأدمن (الاسم قديم)
└── src/
    ├── server.js               ← نقطة البداية: إعدادات ← MongoDB ← تشغيل
    ├── app.js                  ← الـ middleware وتوصيل كل الراوتس
    ├── middleware/auth.js      ← requireAuth: التأكد من توكن الأدمن
    ├── lib/
    │   ├── mongo.js            ← الاتصال + كل الـ models
    │   ├── mailer.js           ← إيميلات الإشعار
    │   └── keepAlive.js        ← السيرفر يزور نفسه كل 10 دقايق
    └── routes/
        ├── projects.js, skills.js, youtube.js  ← قراءة عامة
        ├── services.js, testimonials.js        ← قراءة عامة (+ فورم رأي العميل)
        ├── about.js, settings.js, cv.js        ← قراءة عامة لمستند واحد
        ├── contact.js          ← فورم التواصل العام
        ├── auth.js             ← تسجيل الدخول
        └── admin.js            ← كل عمليات التعديل (محمية بالتوكن)
```

كل ملف فيه شرح

فتحت أي ملف في `server/src` هتلاقي فوقه تعليق بيشرح هو بيعمل إيه ومين بيستخدمه – اقراه الأول قبل الكود.

4 أساسيات HTTP و REST

كل كلام بين الواجهة والسيرفر "طلب" (Request) و"رد" (Response). الطلب له **method** بيقول عايز تعمل إيه، و**عنوان** بيقول على إيه:

Method	معناه	مثال من المشروع
`GET`	هات بيانات	`GET /api/projects`
`POST`	أضف جديد	`POST /api/admin/services`
`PUT`	عدّل/استبدل	`PUT /api/admin/services/:id`
`PATCH`	عدّل جزء صغير	`PATCH /api/admin/messages/:id` (مقروءة/مش مقروءة)
`DELETE`	امسح	`DELETE /api/admin/services/:id`

والرد بيرجع معاه **رقم حالة** (status code):

الكود	معناه	إمتى بيحصل عندنا
`200`	تمام	قراءة أو تعديل ناجح
`201`	اتعمل جديد	إضافة ناجحة، رسالة Contact اتبعتت
`204`	تمام ومفيش محتوى	بعد الحذف
`400`	الطلب غلط	حقل ناقص، نص طويل، رابط مش http
`401`	مش مسموح	من غير توكن، أو باسورد غلط
`404`	مش موجود	طلب CV ومفيش CV مرفوع
`429`	طلبات كتير	تخطّي حد الـ rate limit
`500`	مشكلة في السيرفر	خطأ غير متوقع (بيتسجّل في الـ logs)

REST = طريقة منظمة لتسمية العناوين: كل "مورد" (resource) ليه عنوان (`/services`) والـ method بيحدد العملية. عشان كده كل موارد الأدمن نفس الشكل بالظبط (الفصل 9).

5 رحلة الطلب في app.js

الـ **middleware** = دالة بيعدّي عليها الطلب قبل ما يوصل للراوت. في `app.js` الترتيب كده:

1. Render – `app.set("trust proxy", 1)` فيه وسيط (proxy) قدام السيرفر؛ ده بيخلّي السيرفر يشوف IP الزائر الحقيقي.
2. `helmet()` – يضيف headers أمان لكل رد.
3. `cors(...)` – يسمح لموقعك بس إنه يكلم السيرفر من المتصفح.
4. `express.json()` – يحوّل جسم الطلب (JSON) لـ `req.body`. (رفع الـ CV له حد أكبر: 6MB.)
5. **الراوترات** – أول واحد عنوانه مطابق بيرد.
6. **معالج الأخطاء** في الآخر – أي خطأ يتسجّل ويرجع `{ error }` بكود مناسب (خطأ تحقق = 400).

راوت كامل صغير – server/src/routes/services.js

```
router.get("/", async (req, res) ⇒ {
  res.json(await Service.find({ visible: true }).sort({ order: 1, createdAt: 1 }));
});
```

مفيش `express-async-errors` – `try/catch` بيبعت أي خطأ لمعالج الأخطاء لوحده.

6 قاعدة البيانات (Mongoose)

MongoDB بتخزن **مستندات** (شبه كائنات JSON) جوه **collections**. Mongoose بيخلّينا نحدد شكل كل مستند بـ **Schema**، ونتعامل معاه بـ **Model**:

مختصر — server/src/lib/mongo.js

```
const serviceSchema = new mongoose.Schema({
  titleEn: { type: String, default: "", maxlength: 120 },
  titleAr: { type: String, default: "", maxlength: 120 },
  icon:    { type: String, enum: SERVICE_ICONS, default: "web" },
  visible: { type: Boolean, default: true },
  order:   { type: Number, default: 0 },
}, { timestamps: true });            // createdAt / updatedAt لوحدهم

const Service = mongoose.model("Service", serviceSchema);
await Service.find();  Service.create({...});  Service.findByIdAndDelete(id);
```

الـ collections الموجودة

Model	فيه إيه
`Project`	المشاريع + دراسة الحالة (مشكلة/حل/نتيجة) + tags
`Skill`	المهارات (نسبة، مجموعة، لون)
`YoutubeVideo`	فيديوهات قسم يوتيوب
`Service`	الخدمات (أيقونة، ظاهر/مخفي)
`Testimonial`	آراء العملاء (تقييم، ظاهر، `pending` للي مستني موافقتك)
`About`	مستند واحد: نصوص قسم About + رابط الفيديو
`SiteSettings`	مستند واحد: Hero + الروابط + المكان + الفوتر
`CvFile`	ملف الـ CV نفسه (Buffer) — لأن قرص Render بيتمسح مع كل نشر
`ContactMessage`	رسايل فورم التواصل
`AdminUser`	حساب الأدمن (الباسورد مشفّر)

قواعد في كل الموديلات

- **نسختين لكل نص ظاهر:** `En` و`Ar`...

- **التحقق جوه الـ Schema** (`validate` , `enum` , `maxlength`) – والتعديلات بتستخدم `runValidators: true` عشان تتحقق هي كمان.
- **`_id` بيتحول لـ `id`** قبل الإرسال (transform في آخر `mongo.js`).

7 تسجيل الدخول (JWT)

1. الداشبورد يبعت `POST /api/auth/login` بالإيميل والباسورد.
2. `routes/auth.js` يدوّر على الأدمن بالإيميل، ويقارن الباسورد بالنسخة المشفّرة بـ `bcrypt.compare` .
3. لو صح: يعمل **توكن JWT** موقّع بـ `JWT_SECRET` وصالح 7 أيام، ويرجّعه.
4. الداشبورد يبعت التوكن مع كل طلب: `Authorization: Bearer <token>` .
5. `requireAuth` (`middleware/auth.js`) يتأكد من التوقيع. ولإن `admin.js` فيه `router.use(requireAuth)` في أوله، **ولا راوت أدمن** يتوصل له من غير توكن سليم.

> **ليه الباسورد مش متخزن؟**
>
> بيتخزن **hash** بس (نتيجة تشفير باتجاه واحد). حتى لو حد شاف قاعدة البيانات، ما يقدرش يعرف الباسورد منه.

> **ليه نفس رسالة الخطأ؟**
>
> لو الإيميل غلط أو الباسورد غلط، الرد واحد: "Invalid credentials" – عشان ما نقولش لحد مين الإيميلات اللي موجودة.

8 الحماية وأسبابها

الحماية	بتعمل إيه	ليه
`helmet()`	headers أمان على كل رد	يمنع حيل في المتصفح (iframe، تخمين نوع الملف…) ويخبّي إن السيرفر Express
CORS	موقعك بس يقدر يطلب من المتصفح	أي موقع تاني ما يقدرش يستخدم الـ API بتاعك من صفحات زواره
Rate limit – الدخول	10 محاولات / 15 دقيقة / IP	يمنع تجربة آلاف الباسوردات (brute force)
Rate limit – Contact	10 رسايل / 15 دقيقة / IP	يمنع إغراقك بالسبام
Rate limit – فورم الآراء	5 إرسالات **ناجحة** / ساعة / IP	العميل اللي بيغلط في الكتابة ما يتحظرش
Honeypot	حقل مخفي في فورم الآراء	البوتات بتملاه، فبنتجاهلها بهدوء
قايمة حقول مسموحة	نسخ الحقول المعروفة بس من `req.body`	محدش يقدر يكتب حقول غريبة في قاعدة البيانات
التحقق من الروابط	لازم تبدأ بـ `http(s)://`	يمنع روابط `javascript:` اللي ممكن تنفّذ كود في الصفحة
رفع الـ CV	PDF حقيقي (أول بايتات `%PDF-`) وأقل من 4MB	ما نثقش في امتداد الملف أو كلام المتصفح
`trust proxy`	IP الزائر الحقيقي	من غيره كل الزوار "IP واحد" والحدود تتحسب عليهم مع بعض

> **قرار اتاخد**
>
> الخطة الأصلية كانت توقيف helmet والـ rate limiting مؤقتًا لتبسيط الكود. اتلغى القرار ده: فورم التواصل وتسجيل الدخول كانوا منشورين للناس فعلًا، وتوقيفهم كان هيكشف الموقع. بدل كده، اتشرحوا بتعليقات في الكود.

9 كل عناوين الـ API

عامة (من غير دخول)

الطلب	بيعمل إيه
`GET /api/health`	"أنا شغّال" – بيستخدمه Render والـ keep-alive
`GET /api/projects` · `/skills` · `/youtube`	القوايم مرتبة بـ `order`
`GET /api/services` · `/testimonials`	الظاهر بس (`visible: true`)
`GET /api/about` · `/settings`	المستند المحفوظ أو `{ exists: false }`
`GET /api/cv/info` · `GET /api/cv`	هل فيه CV؟ / تحميل الملف
`POST /api/contact`	رسالة من فورم التواصل
`POST /api/testimonials/submit`	رأي عميل من اللينك (بيتحفظ Pending)
`POST /api/auth/login`	تسجيل الدخول ← توكن

الأدمن (لازم توكن) – تحت `/api/admin`

المورد	العمليات
`/services` · `/youtube` · `/skills` · `/projects` · `/testimonials`	`GET` الكل · `POST` إضافة · `PUT /:id` تعديل · `DELETE /:id` حذف
`/settings` · `/about`	`PUT` · `GET` حفظ المستند · `DELETE` رجوع للأصلي
`/cv`	`PUT` رفع/استبدال · `DELETE` (PDF (base64
`/messages`	`PATCH /:id` · `GET` مقروءة · `DELETE /:id`

> **جرّب بنفسك**
>
> افتح في المتصفح `https://salem-portfolio-server.onrender.com/api/projects` – هتشوف الـ JSON اللي الموقع بيستخدمه.

10 الإيميل وإبقاء السيرفر صاحي

إيميلات الإشعار (`lib/mailer.js`)

لما حد يبعت رسالة، السيرفر **يرد على الزائر الأول** وبعدين يبعت الإيميل في الخلفية – فلو الإيميل اتأخر أو فشل، الزائر ما يستناش والرسالة متخزنة في الداشبورد على أي حال.

> **حالة الإيميل حاليًا**
>
> خطة Render المجانية بتقفل بورتات SMTP، فالإشعار ما بيوصلش من السيرفر المنشور. الحل جاهز (خدمة Resend عبر HTTP) على branch اسمه `feat/email-resend` ، ومؤجّل بقرارك. الرسايل والآراء بتوصل في الداشبورد عادي.

(Keep-alive (`lib/keepAlive.js`

Render المجاني بينيّم السيرفر بعد 15 دقيقة من غير زيارات، وأول زيارة بعدها تستنى حوالي دقيقة. الحل: السيرفر يزور `/api/health` بتاعه هو كل **10 دقايق**، فما يوصلش لـ 15 دقيقة أبدًا.

- بيشتغل على Render بس (بيعرف عنوانه من `RENDER_EXTERNAL_URL`)، ومش بيشتغل على جهازك.
- للإيقاف: متغير `KEEP_ALIVE=off` على Render.
- سيرفر صاحي طول الشهر بياخد ~744 ساعة من 750 ساعة مجانية.

11 النشر على Render و Atlas

Render (السيرفر)

- Web Service مربوط بالريبو، Start = `npm start`، Build = `npm install`، **Root Directory** = `server`.
- **أي push على** Render = `main` يبني وينشر لوحده (2-3 دقايق).
- الإعدادات السرية في **Environment** (مش في git أبدًا).
- لو حاجة باظت: **Logs** فيها كل `console.log` و `console.error`.

MongoDB Atlas (قاعدة البيانات)

- Cluster مجاني M0، ومستخدم قاعدة بيانات بباسورد.
- **Network Access** لازم فيه `0.0.0.0/0` – لأن عنوان Render بيتغير.
- رابط الاتصال: `mongodb+srv://USER:PASSWORD@cluster0.xxxx.mongodb.net/salem_portfolio` – اسم القاعدة قبل `?`.

> **مؤجّل بقرارك**
>
> باسورد Atlas و `JWT_SECRET` وباسورد الأدمن اتكتبوا في محادثة قبل كده. المفروض يتغيروا قبل أي إطلاق كبير (الخطوات في `ROADMAP.md`).

12 إضافة مورد جديد

مثال: "الشهادات" (Certificates). الجزء الخاص بالسيرفر:

1. **Schema + Model** في `lib/mongo.js` وضيفه في `models` :

```
const certificateSchema = new mongoose.Schema({
  titleEn: { type: String, default: "", maxlength: 160 },
  titleAr: { type: String, default: "", maxlength: 160 },
  issuer:  { type: String, default: "", maxlength: 120 },
  url:     { type: String, default: "", maxlength: 300 },
  order:   { type: Number, default: 0 },
}, { timestamps: true });
```

2. **راوت عام** `routes/certificates.js` (انسخ `services.js`) + سطرين في `require` و `app.js` (`app.use("/api/certificates", ...)`).

3. **راوتس الأدمن** في `admin.js` (انسخ قسم Services): قايمة حقول مسموحة + GET/POST/PUT/DELETE.

4. **جرّب بـ curl أو من الداشبورد** على جهازك، وتأكد إن البيانات الغلط بترجع 400.

5. بعدها الجزء الخاص بالواجهة (موجود في دليل الفرونت إند، الفصل 10).

13 مشاكل حصلت واتعلمنا منها

المشكلة	السبب	الحل / الدرس
Deploy فشل على Render	Atlas كان سامح لـ IP جهازك بس	`0.0.0.0/0` في Network Access
الموقع ما يقدرش يكلم السيرفر	`CLIENT_ORIGIN` على اسم الحساب القديم (solx4)	لازم يطابق رابط GitHub Pages بالظبط
فورم التواصل بيحمّل للأبد	السيرفر بيستنى الإيميل، وبورت SMTP مقفول على Render	الرد للزائر الأول، والإيميل في الخلفية بمهلة 10 ثواني
حدود الـ rate limit على كل الزوار مع بعض	من غير `trust proxy` كل الطلبات من IP الوسيط	`app.set("trust proxy", 1)`
عميل غلط 5 مرات فاتحظر	الحد كان بيحسب الطلبات المرفوضة	`skipFailedRequests: true`
خطأ تحقق بيرجع 500	معالج الأخطاء ما كانش بيفرّق	`ValidationError` ← 400
"Invalid credentials" بالبيانات الصح	الـ `.env` اتغير بعد الـ seed	تغيير بيانات الأدمن لازم يتحدّث في قاعدة البيانات كمان

14 قاموس المصطلحات

المصطلح	معناه ببساطة
API	مجموعة عناوين على السيرفر الواجهة بتكلمها.
Endpoint / Route	عنوان واحد + method، زي `GET /api/projects`.
Middleware	دالة بيعدّي عليها الطلب قبل الراوت.
JSON	شكل نصي للبيانات بيتبادلوا بيه الواجهة والسيرفر.
Schema / Model	شكل المستند وقواعده / الأداة اللي بتقرا وتكتب بيها.
Collection	مجموعة مستندات من نوع واحد (زي جدول).
JWT	توكن موقّع بيثبت إنك الأدمن.
Hash (bcrypt)	تشفير باتجاه واحد للباسورد.
Rate limit	حد لعدد الطلبات من نفس الجهاز في فترة.
CORS	قاعدة في المتصفح: مين مسموح له يطلب من السيرفر.
Environment variable	إعداد خارج الكود (أسرار، عناوين).
Deploy	نشر نسخة جديدة على السيرفر الحقيقي.
Proxy	وسيط بين الزائر والسيرفر.
Cold start	صحيان السيرفر بعد ما نام.

الدليل ده بيوصف الكود في أكتوبر 2026. الملف المصدر `docs/guides/backend-guide.html` – عدّله وأعد الطباعة بـ `docs/guides/build-pdfs.sh`.